

1.3. Chemical Nanocomputer

In general terms a chemical computer is one that processes information in by making and breaking chemical bonds and it stores logic states or information in the resulting chemical (i.e., molecular) structures. In a chemical nanocomputer computing, would be based on chemical reactions (bond breaking and forming) and the inputs will be encoded in the molecular structure of the reactants and outputs extracted from the structure of the products. In order to create a chemical nanocomputer, engineers need to be able to control individual atoms and molecules so that these atoms and molecules can be made to perform controllable calculations and data storage tasks. The development of a true chemical nanocomputer will likely proceed along lines similar to genetic engineering.



The future of biological Nanocomputers

1.4. Biological Nanocomputer

In a manner of speaking biological Nanocomputer already exist in Nature. All we need to do is figure out a way to replicate it. These naturally occurring systems are largely uncontrollable by humans and that makes artificial fabrication or implementation of this category of “natural” biochemically based computers seems far off because the mechanisms for animal brains and nervous systems still are poorly understood.

1.5. DNA Nanocomputer

In 1994, Leonard Adelman took a giant step towards a different kind of chemical or artificial biochemical computer when he used fragments of DNA to compute the solution to a complex graph theory problem. DNA nanocomputers would use DNA to store information and perform complex calculations. DNA has a vast amount of storage capacity that enables it to hold the complex blueprints of living organisms. The storage capacity of a single gram of DNA can hold as much information as one trillion compact discs.

1.6. Quantum Nanocomputer

A quantum computer uses quantum mechanical phenomena, such as entanglement and superposition to process data. Quantum computation aims to use the quantum properties of particles to represent and structure